Relatório do Trabalho: Desenvolvimento e Evolução da Interface Web da Câmara Municipal de Buriti do Codô

Gustavo Antônio de Sousa[1], **Luan Cristian da Silva Rufino**[1], **Nilson Bruno dos Santos Lima**[1], **Rian de Oliveira Leal**[1]

[1]Curso de Sistemas de Informação – Universidade Federal do Piauí (UFPI)
Picos – PI – Brasil

gustavo.antonio@ufpi.edu.br, luan.rufino@ufpi.edu.br

nilson.lima@ufpi.edu.br, rian.leal@ufpi.edu.br

Abstract. *This document presents the complete report on the development of the institutional web portal for the fictional City Council of Buriti do Codô (Câmara Municipal de Buriti do Codô), created as a practical project for the Web Programming course. The structural and visual foundation was built strictly with native HTML5 and CSS3, applying semantic markup, web accessibility (a11y), Box Model styling, forms, and responsive layouts with CSS Flexbox, CSS Grid Layout, and Media Queries. In its complete state, the system operates as an interactive web application powered by native JavaScript (ECMAScript), incorporating dynamic DOM manipulation, event-driven programming, section-based navigation, dynamic template pages (`noticia.html` and `vereador.html`) fed via URL parameters (`URLSearchParams`), real-time search and filtering, dynamic element creation and removal, and client-side data persistence using five keys in `localStorage`, five keys in `sessionStorage`, and five HTTP cookies.*

Resumo. *Este documento apresenta o relatório completo do desenvolvimento do portal institucional da fictícia Câmara Municipal de Buriti do Codô, concebido como trabalho prático da disciplina de Programação para a Web. A base visual e semântica do projeto foi estruturada utilizando estritamente HTML5 e CSS3 nativos, contemplando estruturação semântica, acessibilidade web (a11y), estilização baseada no Box Model, formulários e layouts responsivos com CSS Flexbox, CSS Grid Layout e Media Queries. Em seu estado atual, o sistema funciona como uma aplicação web interativa impulsionada por JavaScript nativo, integrando manipulação dinâmica da árvore DOM, tratamento de eventos, navegação entre seções sem recarregamento, páginas internas dinâmicas (`noticia.html` e `vereador.html`) alimentadas por parâmetros de URL (`URLSearchParams`), filtros de busca em tempo real, criação e remoção dinâmica de elementos e persistência de dados no navegador por meio de cinco chaves em `localStorage`, cinco chaves em `sessionStorage` e cinco cookies HTTP.*

1. Informações Gerais e Descrição do Projeto

O projeto consiste em uma aplicação web criada para simular o portal institucional e de transparência da **Câmara Municipal de Buriti do Codô**, um município fictício idealizado pelo grupo para atender ao tema central proposto na disciplina de Programação

para a Web (*Câmara de Vereadores*). Para contextualizar a simulação com o curso, os próprios quatro integrantes da equipe figuram como os parlamentares da Casa Legislativa fictícia, filiados a partidos temáticos alusivos à área de computação (`SI`, `TI`, `HTML` e `CSS`). A interface foi planejada para simular o acesso do cidadão às informações do Poder Legislativo municipal, adotando uma identidade visual institucional com brasão próprio, imagem panorâmica no cabeçalho e uma paleta de cores em tons de Verde (`#005637`), Azul (`#002496`) e Amarelo/Ouro (`#FFB612`) combinada com fundos claros e tipografia legível. Todas as seções institucionais foram estruturadas utilizando HTML5 semântico e CSS3 responsivo, contemplando a apresentação da Câmara, o catálogo de vereadores em cartões (*cards*), o mural de notícias legislativas, a tabela de sessões plenárias, os dados de contato e o formulário da Ouvidoria Parlamentar.

No estado atual da aplicação, o portal opera como um sistema web dinâmico, interativo e com persistência de estado real no navegador utilizando a linguagem **JavaScript** (`script.js`). O cabeçalho conta com um painel suspenso de configurações de acessibilidade (com alternância entre Modo Claro e Modo Escuro, ajuste do tamanho da fonte entre `12px` e `22px` e botão de restauração rápida), além de navegação inteligente entre seções pela barra lateral (com suporte a âncoras de URL `#secao` vindas de páginas internas) e um botão para ocultar ou exibir o menu lateral, ampliando a área de leitura.

A bancada parlamentar fictícia reúne os quatro vereadores da Mesa Diretora (`Luan Cristian`, `Gustavo Antonio`, `Rian Leal` e `Nilson Bruno`), integrando busca instantânea por nome, filtro por partido, contador dinâmico de resultados, botão de limpeza de filtros, painel de resumo rápido no próprio `index.html` (`Ver detalhes`) e uma página coringa dedicada (`vereador.html?id=...`) acionada pelo botão `Ver página`, que renderiza o perfil completo do parlamentar e a listagem dos demais vereadores. De modo análogo, as seções de Notícias e Sessões contam com pesquisa em tempo real, filtro por tipo de sessão, contadores dinâmicos e uma página coringa de leitura (`noticia.html?id=...`) acionada pelo botão `Ler mais`, que monta dinamicamente a matéria completa e sugere outras notícias. Por fim, o formulário da Ouvidoria Parlamentar dispõe de validação de campos, contador de caracteres em tempo real (`0 / 300`), botão de limpeza, preenchimento automático via *cookies* e criação dinâmica de cartões de mensagens enviadas com opção de exclusão individual, totalizando quinze chaves de armazenamento no navegador (cinco em `localStorage`, cinco em `sessionStorage` e cinco `cookies` HTTP).

2. Como os Requisitos Foram Atendidos

Abaixo, explicamos e justificamos como cada um dos requisitos técnicos e conceituais exigidos ao longo de toda a disciplina (Aulas 02 a 15) foi implementado na prática dentro do código.

2.1. HTML Semântico, Acessibilidade e Formulários (Aulas 02, 03 e 04)

Conforme os conceitos introduzidos nas primeiras aulas da disciplina, a estrutura dos documentos (`index.html`, `noticia.html` e `vereador.html`) substituiu o uso genérico de contêineres `<div>` por *landmarks* semânticos que organizam a árvore do DOM (*Document Object Model*): `<header>`, `<nav>`, `<main>`, `<section>`, `<article>` e `<footer>`. Para garantir boas práticas de acessibilidade (*a11y*), implementamos o

link oculto de classe `.pular-conteudo` (*Skip to Content*) que se torna visível ao receber foco via teclado (`:focus`), atributos `aria-label="Menu principal"`, textos alternativos descritivos (`alt`) em todas as fotos de perfil, cabeçalhos de tabela com `scope="col"` e marcação semântica de datas com `<time datetime="...">`.

Na seção de Ouvidoria, o formulário (`#form-ouvidoria`) utiliza o agrupamento `<fieldset>` com `<legend>` e amarração explícita entre cada `<label for="...">` e o respectivo campo de entrada (`<input type="text">`, `<input type="email">`, `<select>` e `<textarea maxlength="300">`). Essa estrutura é integrada ao JavaScript com botões de envio e limpeza (`#btn-limpar-ouvidoria`), parágrafo de feedback (`#feedback-ouvidoria`) e painel dinâmico de mensagens registradas (`#painel-protocolos`).

2.2. CSS, Box Model, Flexbox, Grid e Modo Escuro (Aulas 05, 06 e 07)

A estilização (`style.css`) foi construída utilizando variáveis CSS no seletor `:root` (`--cor-principal: #005637`, `--cor-secudaria: #002496` e `--cor-terciaria: #FFB612`) e aplicação global de `box-sizing: border-box` no seletor universal (`*`). A arquitetura espacial combina **CSS Flexbox** no cabeçalho (`.cabecalho-container`), na classe `.pagina-container` (posicionando a `sidebar` com `position: sticky` ao lado do conteúdo principal) e na estrutura `.perfil-vereador` das páginas internas, além de **CSS Grid Layout** nas classes `.lista-vereadores` e `.grade-informativa` com `grid-template-columns: repeat(auto-fit, minmax(...))` e regras de responsividade (`@media`) para telas de até `768px` e `600px`.

Os cartões dos vereadores são padronizados com `display: flex` e `flex-direction: column`, aplicando `margin-top: auto` aos botões de ação e estilizando links em formato de botão (`.btn-link` e `.btn-pagina`). O arquivo também contempla os estilos dedicados às páginas internas de notícia e de vereador (`.link-voltar`, `.pagina-noticia`, `.pagina-vereador`, `.categoria-noticia`, `.resumo-noticia`, `.perfil-foto` com `aspect-ratio: 3 / 4`, `.perfil-info`, `.perfil-cargo`, `.outras-noticias` e `.outros-vereadores`), bem como a classe `.tema-escuro` aplicada ao `<body>` e as classes de estado `.oculto`, `.sidebar-recolhida`, `.card-ativo`, `.link-ativo` e `.bloco-destaque`.

2.3. Fundamentos de JavaScript, Controle de Fluxo e Funções (Aulas 09, 10 e 11)

A camada lógica desenvolvida em `script.js` aplica de forma integrada as estruturas da linguagem JavaScript estudadas nas Aulas 09 a 11:

- **Variáveis e Operadores:** Utilizamos `const` para referências de elementos do DOM e funções, e `let` para variáveis de controle mutáveis (`vereadorAbertoAtual`, `secaoSalva`, `visiveis`, `exibidas` e `tamanhoAtual`). Empregamos o operador de coalescência nula (`??`) para valores padrão de armazenamento, o operador ternário (`? :`) para alternar textos de botões em tempo real e o operador `instanceof Node` em `criarLinhaDado()`.

- **Condicionais (`if/else` e `switch`):** Enquanto blocos `if/else` controlam a validação do formulário, a verificação de páginas internas e a filtragem combinada, a estrutura `switch` foi implementada na função `obterRotuloAssunto(valorAssunto)` para converter os códigos do seletor da Ouvidoria (`duvida`, `sugestao`, `reclamacao`, `elogio`) em rótulos formatados.
- **Laços de Repetição (`for`, `for...of`, `for...in`, `break` e `continue`):** O laço `for...of` percorre coleções de nós do DOM e vetores de dados (`dadosVereadores` e `dadosNoticias`); o comando `break` encerra a busca em `buscarVereadorPorNome()` assim que o parlamentar é encontrado; o comando `continue` avança para a próxima iteração em `filtrarNoticias()`, `filtrarTabelaSessoes()`, `renderizarPaginaNoticia()` e `renderizarPaginaVereador()`; e o laço `for...in` percorre dinamicamente as chaves do objeto `camposMensagem` ao montar o cartão na lista de mensagens enviadas.
- **Declarações de Funções e Parâmetros:** O código utiliza declarações tradicionais (`function`), expressões de função atribuídas a constantes (`salvarLocal` e `salvarSessao`), *Arrow Functions* (`obterLocal`, `obterSessao` e predicados do método `find()`), parâmetros com valor padrão (`passo = 2`, `dias = 7`, `texto = ""`, `classe = ""`) e a combinação de parâmetro *rest* (`...partes`) com o objeto `arguments` na função `formatarConfirmacaoEnvio(saudacao, ...partes)`.

2.4. Objetos, Arrays, Strings, Datas e Parâmetros de URL (Aula 12)

Seguindo o conteúdo da Aula 12, o acervo fictício detalhado dos parlamentares e das notícias foi estruturado nos vetores de objetos literais `dadosVereadores` e `dadosNoticias`. Em `dadosVereadores`, cada objeto encapsula as propriedades `nome`, `id`, `foto`, `partido`, `cargo`, `mandato`, `comissao`, `gabinete`, `biografia` e o array interno `projetos` contendo os Projetos de Lei do respectivo vereador. Em `dadosNoticias`, cada objeto armazena `id`, `titulo`, `resumo`, `rotulo`, `detalhe`, `iso`, `data` e `cat`. De forma complementar, o evento de envio da Ouvidoria instancia o objeto literal `camposMensagem` agrupando os dados do remetente e o texto da manifestação.

Para busca em vetores e manipulação textual, temporal e de endereços, aplicamos o método de array `find()` para localizar registros pelo `id`, os métodos nativos de `String` (`toLowerCase()`, `trim()`, `indexOf()`, `startsWith()`, `replace()`, `split()` e `substring()`), a classe nativa `Date` (`new Date().toLocaleDateString("pt-BR")`) e a interface `URLSearchParams(window.location.search)` combinada com `window.location.hash` e `history.replaceState()` para leitura de parâmetros nas páginas internas e navegação entre seções.

2.5. Manipulação Dinâmica da Árvore DOM (Aula 13)

Com base nos conceitos da Aula 13, a interface é atualizada dinamicamente através dos métodos nativos do DOM:

- **Seleção e Leitura de Atributos:** Utilizamos `document.querySelector()` e `document.querySelectorAll()` (incluindo seletores de atributo como `a[href^="#"]`), combinados com `getAttribute()`, `setAttribute()` e `dataset.pagina` (`data-pagina="interna"`) para ler atributos customizados diretamente dos elementos HTML.
- **Atualização de Conteúdo, Classes e Estilos:** Adotamos `textContent` e `document.title` para atualizar contadores, mensagens e o título da aba do navegador com segurança, `style.fontSize` para redimensionamento tipográfico e a API `classList` (`add`, `remove`, `toggle` e `contains`) para gerenciar a visibilidade e o tema da aplicação.
- **Criação e Remoção Dinâmica de Elementos:** Através de `document.createElement()`, `document.createTextNode()` e `appendChild()` (encapsulados também nas funções construtoras `criarElemento`, `criarLinkVoltar`, `criarBlocoInfo` e `criarLinhaDado`), o sistema constrói em memória o painel rápido `#painel-vereador`, os cartões da Ouvidoria em `#lista-protocolos` e todo o conteúdo das páginas internas `noticia.html` (`#conteudo-noticia`) e `vereador.html` (`#conteudo-vereador`). Na Ouvidoria, o método `remove()` exclui o cartão da árvore DOM quando o usuário aciona o botão de exclusão.

2.6. Eventos, Controle de Propagação e Delegação (Aula 14)

Apoiando-se nos conceitos da Aula 14, todos os fluxos interativos foram registrados com `addEventListener`:

- **Diversidade de Eventos:** Implementamos ouvintes para `DOMContentLoaded` (inicialização de preferências, sessão e cookies), `click` (botões de ação, cartões e navegação), `input` (busca em tempo real de vereadores e contador de caracteres da Ouvidoria), `keyup` (pesquisa de notícias por palavra-chave), `change` (seletores de partido e de tipo de sessão), `submit` (envio validado do formulário) e `mouseover` / `mouseout` (destaque interativo nos cartões informativos `.bloco-info`).
- **Controle de Propagação e Comportamento Nativo:** No menu suspenso de Configurações, aplicamos `event.stopPropagation()` no botão e no painel interno para impedir que o clique chegue ao `document`, permitindo que um ouvinte global no `document` feche o painel automaticamente ao clicar em qualquer área externa. Na `sidebar`, o ouvinte verifica se o link clicado aponta para uma seção existente na própria página (`destino.startsWith("#") && secaoExiste(...)`) antes de acionar `event.preventDefault()`, permitindo que links entre páginas (como `index.html#noticias`) sigam seu fluxo normal.
- **Delegação de Eventos:** Nos contêineres `#container-vereadores`, `.sidebar` e `#lista-protocolos`, anexamos um único ouvinte no elemento pai e utilizamos `event.target.closest()` e `event.target.matches()` para capturar cliques em botões internos ou gerados dinamicamente.

2.7. Persistência no Navegador: Web Storage e Cookies HTTP (Aula 15)

Para atender integralmente à exigência de persistência de dados no cliente (Aula 15), o projeto implementa cinco chaves em cada um dos três mecanismos de armazenamento do navegador (totalizando 15 registros):

- **Armazenamento Permanente (`localStorage` – 5 chaves):** `temaEscuro` (guarda `"sim"` ou `"nao"` para o Modo Escuro), `tamanhoFonte` (tamanho da fonte em pixels entre `12` e `22`), `totalVisitas` (contador incremental de acessos ao portal), `ultimoAcesso` (data formatada da última visita) e `ultimoVereadorConsultado` (nome do último parlamentar visualizado, exibido no rodapé).
- **Armazenamento de Sessão (`sessionStorage` – 5 chaves):** `secaoAtiva` (mantém a seção selecionada na `sidebar`, preservando o estado ao navegar pelas páginas internas), `termoBuscaNoticia` (preserva o texto pesquisado no mural de notícias), `filtroTipoSessao` (guarda a categoria selecionada na tabela de sessões), `noticiasMarcadas` (registro de leitura de notícias na sessão) e `cliquesSessao` (total de navegações realizadas no menu lateral durante a sessão).
- **Cookies HTTP (`document.cookie` – 5 cookies):** Gerenciados com `encodeURIComponent`, `decodeURIComponent` e tempo de expiração `max-age` de 7 dias: `cidadaoNome` (nome do cidadão para autopreenchimento na Ouvidoria), `cidadaoEmail` (e-mail do cidadão), `ultimoAssuntoOuvidoria` (última categoria de assunto escolhida), `dataUltimoEnvio` (data da última manifestação registrada) e `totalManifestacoesEnviadas` (contador acumulado de envios).

2.8. Documentação dos Elementos do DOM Manipulados

Para cumprir a exigência documental do projeto, apresentamos abaixo a lista detalhada contendo os nomes das constantes/variáveis instanciadas em `script.js`, o seletor correspondente nos arquivos HTML e a respectiva função na interface:

No Cabeçalho, Menu Lateral e Estrutura Geral:

- `corpoPagina` (`document.body`): recebe a classe `.tema-escuro`, o tamanho de fonte dinâmico e o atributo `data-pagina="interna"` nas páginas secundárias.
- `btnConfig` (`#btn-config`) e `painelConfig` (`#painel-config`): botão do cabeçalho e menu suspenso que abriga as opções de acessibilidade visual.
- `btnTema` (`#btn-tema`): alterna entre o Modo Claro e o Modo Escuro.
- `btnDiminuirFonte` (`#btn-diminuir-fonte`), `btnAumentarFonte` (`#btn-aumentar-fonte`) e `valorFonte` (`#valor-fonte`): botões de ajuste tipográfico (`A-` e `A+`) e rótulo indicador em pixels (`16px`).
- `btnRestaurarConfig` (`#btn-restaurar-config`): botão que restaura imediatamente o tema claro e a fonte padrão.
- `btnMenu` (`#btn-menu`) e `menuLateral` / `menuSidebar` (`.sidebar`): botão que recolhe ou expande a aba de seções (`.sidebar-recolhida`) e barra de navegação lateral.

- `secoesPagina` (`main section`): coleção das seções institucionais exibidas ou ocultadas dinamicamente no `index.html`.
- `blocosInfo` (`.bloco-info`): cartões informativos que recebem destaque visual nos eventos `mouseover` e `mouseout`.
- `infoAcesso` (`#info-acesso`): parágrafo no rodapé que exibe o resumo de visitas, último acesso e último parlamentar consultado.

Na Seção de Vereadores (`#vereadores`) e em `vereador.html`:

- `campoBuscaVereador` (`#busca-vereador`): campo de texto para busca instantânea de parlamentares pelo nome.
- `seletorPartido` (`#filtro-partido`): seletor para filtragem por partido (`SI`, `TI`, `HTML`, `CSS` ou `todos`).
- `btnLimparVereadores` (`#btn-limpar-vereadores`): botão que limpa os filtros aplicados e restaura a listagem completa.
- `contadorVereadores` (`#contador-vereadores`): rótulo dinâmico que mostra a quantidade de vereadores visíveis ou mensagem de nenhum resultado.
- `containerVereadores` (`#container-vereadores`): grade que agrupa os cartões `.card-vereador` (com os botões `.btn-detalhes` e links `.btn-pagina`).
- `painelVereador` (`#painel-vereador`): contêiner onde a ficha resumida do parlamentar e o link para sua página completa são gerados via DOM.
- `#conteudo-vereador` (em `vereador.html`): elemento onde `renderizarPaginaVereador()` constrói o perfil completo do parlamentar e a grade de outros vereadores.

Nas Seções de Notícias (`#noticias`), Sessões (`#sessoes`) e em `noticia.html`:

- `campoBuscaNoticia` (`#busca-noticia`) e `contadorNoticias` (`#contador-noticias`): campo de busca por palavra-chave e contador de notícias exibidas.
- `listaNoticias` (`.lista-noticias`): lista de matérias cujos itens apresentam o resumo e o botão `.btn-ler-noticia` (`Ler mais`) que abre `noticia.html?id=...`.
- `#conteudo-noticia` (em `noticia.html`): elemento onde `renderizarPaginaNoticia()` monta a matéria completa e a seção de outras notícias.
- `seletorSessao` (`#filtro-sessao`), `contadorSessoes` (`#contador-sessoes`) e `linhasSessoes` (`#sessoes tbody tr`): seletor de categoria (`Ordinária` / `Extraordinária`), contador de sessões visíveis e linhas da tabela plenária.

Na Seção de Ouvidoria (`#ouvidoria`):

- `formOuvidoria` (`#form-ouvidoria`): formulário de envio de manifestações parlamentares.
- `campoNome` (`#nome`), `campoEmail` (`#email`), `campoAssunto` (`#assunto`) e `campoMensagem` (`#mensagem`): campos de entrada validados no envio e autopreenchidos via *cookies*.

- `contadorCaracteres` (`#contador-caracteres`): parágrafo que exibe a contagem em tempo real (`0 / 300`) da mensagem.
- `btnLimparOuvidoria` (`#btn-limpar-ouvidoria`): botão secundário para limpar os campos do formulário.
- `feedbackOuvidoria` (`#feedback-ouvidoria`): exibe avisos de validação ou a mensagem de sucesso formatada.
- `painelProtocolos` (`#painel-protocolos`), `contadorProtocolos` (`#contador-protocolos`) e `listaProtocolos` (`#lista-protocolos`): painel, contador dinâmico e lista `<ul>` que armazena os cartões de mensagens enviadas (`.item-protocolo`).

2.9. Documentação dos Arrays, Objetos e Funções Criados

Para atender às especificações de documentação da lógica do sistema, listamos abaixo todos os objetos, vetores e funções implementados em `script.js`:

Estruturas de Dados (Arrays e Objetos):

- `dadosVereadores`: Array contendo 4 objetos literais correspondentes aos vereadores fictícios interpretados pelos membros da equipe (Luan Cristian, Gustavo Antonio, Rian Leal e Nilson Bruno), contendo as propriedades `nome`, `id`, `foto`, `partido`, `cargo`, `mandato`, `comissao`, `gabinete`, `biografia` e o array `projetos`.
- `dadosNoticias`: Array contendo 6 objetos literais correspondentes às matérias legislativas fictícias do portal, contendo as propriedades `id`, `titulo`, `resumo`, `rotulo`, `detalhe`, `iso`, `data` e `cat`.
- `camposMensagem`: Objeto literal criado no envio do formulário da Ouvidoria contendo as chaves `Remetente` e `Mensagem`, iterado com `for...in` para construção dos elementos do cartão.

Funções de Configurações, Acessibilidade e `localStorage`:

- `salvarLocal(chave, valor)` e `obterLocal(chave, padrao = null)`: Gravador e leitor com valor padrão para o `localStorage`.
- `atualizarTextoBotaoTema()` e `atualizarRodape()`: Sincronizam o texto do botão de tema e as estatísticas de acesso exibidas no rodapé.
- `inicializarPreferencias()`: Inicializa as 5 chaves do `localStorage`, contabiliza a visita atual e aplica o tema e o tamanho de fonte salvos.
- `alterarTamanhoFonte(passo = 2)` e `restaurarConfiguracoesPadrao()`: Modificam o tamanho da fonte respeitando os limites de `12px` a `22px` e redefinem o tema claro e a fonte original de `16px`.

Funções do Catálogo de Vereadores e Manipulação do DOM:

- `buscarVereadorPorNome(nomeProcurado)`: Localiza e retorna o objeto do parlamentar no array `dadosVereadores`.
- `filtrarVereadores()` e `limparFiltrosVereadores()`: Aplicam os filtros combinados de nome e partido nos cartões, atualizam o contador `#contador-vereadores` e redefinem a busca para o estado inicial.

- `atualizarEstadoCartoes(nomeAtivo)` e `renderizarDetalhesVereador(dados)`: Destacam o cartão selecionado e constroem dinamicamente no DOM a ficha resumida, a lista de projetos de lei e o botão de acesso à página completa do parlamentar.

Funções de Navegação, Notícias, Sessões e `sessionStorage`:

- `salvarSessao(chave, valor)` e `obterSessao(chave, padrao = "")`: Funções auxiliares de persistência no `sessionStorage`.
- `secaoExiste(idSecao)`, `mostrarSecao(idSecao)` e `inicializarSessao()`: Verificam a existência da seção na página, controlam a navegação entre seções (incluindo leitura do hash da URL ao retornar das páginas internas), destacam o link ativo na `sidebar` e restauram as 5 chaves do `sessionStorage`.
- `filtrarNoticias()` e `filtrarTabelaSessoes()`: Filtram em tempo real a lista de notícias e as linhas da tabela de sessões plenárias, atualizando os contadores `#contador-noticias` e `#contador-sessoes`.

Funções da Ouvidoria Parlamentar e `cookies` HTTP:

- `definirCookie(nome, valor, dias = 7)` e `lerCookie(nome, padrao = "")`: Codificam, gravam e decodificam cookies HTTP em `document.cookie`.
- `formatarConfirmacaoEnvio(saudacao, ...partes)` e `obterRotuloAssunto(valorAssunto)`: Montam o texto de confirmação do envio (usando parâmetro *rest* e `arguments`) e traduzem o código do assunto via `switch`.
- `atualizarContadorCaracteres()`, `atualizarContadorProtocolos()` e `limparFormularioOuvidoria()`: Atualizam o contador de caracteres (`0 / 300`), contabilizam os cartões de mensagens ativas e limpam os campos do formulário.
- `inicializarCookiesOuvidoria()` e `adicionarMensagemNaLista(tituloCard, camposMensagem)`: Inicializam os 5 cookies HTTP com autopreenchimento do formulário e constroem dinamicamente os cartões de mensagens enviadas com suporte à exclusão individual.

Funções das Páginas Internas Coringa (`noticia.html` e `vereador.html`):

- `lerParametroId()`: Utiliza `URLSearchParams(window.location.search)` para obter o parâmetro `id` da URL.
- `criarElemento(tag, texto = "", classe = "")` e `criarLinkVoltar(texto, destino)`: Funções utilitárias que instanciam elementos do DOM já configurados com texto/classe e criam o link de retorno `.link-voltar`.
- `criarBlocoInfo(titulo)` e `criarLinhaDado(rotulo, valor)`: Construtores auxiliares que montam os blocos informativos e parágrafos rotulados (aceitando texto ou um nó DOM via `instanceof Node`) no perfil do vereador.
- `renderizarPaginaNoticia()` e `renderizarPaginaVereador()`: Buscam o registro pelo `id` em `dadosNoticias` ou `dadosVereadores` usando `find()`, atualizam o `document.title`, constroem toda a página da

notícia ou do vereador (com tratamento para IDs inexistentes) e geram os blocos `Outras notícias` e `Outros vereadores`.

3. Uso de Inteligência Artificial

A Inteligência Artificial atuou de forma controlada durante todo o desenvolvimento do projeto, servindo como ferramenta de auxílio, consulta conceitual, geração de conteúdo textual temático para preenchimento da página e verificação de sintaxe, em conformidade com as regras impostas na disciplina. Os modelos utilizados foram o **Gemini** e o **Claude**.

Tanto na primeira quanto na segunda etapa do trabalho, a IA foi utilizada para criar os textos políticos, institucionais e genéricos fictícios que preenchem e ambientam o portal do município de Buriti do Codô, padronizar os avatares dos integrantes da equipe como vereadores e consultar propriedades específicas de CSS3 e JavaScript que complementavam as notas de aula. Após compreender a lógica explicada pelas ferramentas, o grupo estruturou e implementou o código nos seguintes cenários:

- **Geração dos Textos Políticos e Conteúdos Genéricos para Preenchimento da Página:** Utilizamos o Gemini e o Claude para elaborar os textos genéricos e fictícios de temática legislativa e política utilizados para preencher todas as seções da página da Câmara Municipal de Buriti do Codô (evitando o uso de textos de marcação sem sentido como *Lorem Ipsum*). Isso incluiu os textos institucionais sobre as funções da Câmara, as Comissões Permanentes e o Papel da Mesa Diretora, as biografias e os Projetos de Lei simulados (`PL n° ...`) atribuídos a cada membro do grupo no array `dadosVereadores`, as seis matérias jornalísticas fictícias (com resumos e parágrafos de detalhes técnicos) no array `dadosNoticias`, as pautas de deliberação na tabela de Sessões Plenárias, os dados fictícios de ramais/setores de contato e as instruções de atendimento da Ouvidoria.
- **Padronização Visual das Fotografias dos Vereadores e Identidade Visual:** Na etapa de construção visual do catálogo parlamentar, utilizamos o Gemini para gerar o brasão municipal fictício e editar as fotografias de perfil dos quatro estudantes da equipe, adicionando vestimenta formal (terno e gravata) para caracterizar os autores como vereadores da Câmara fictícia.
- **Alinhamento Proporcional em Flexbox e Modo Escuro:** Consultamos as ferramentas para ajustar o alinhamento vertical interno dos cartões de vereadores com `margin-top: auto` nos botões de ação, estruturar o layout de perfil (`.perfil-vereador`) e selecionar uma paleta de cores de bom contraste visual para as regras da classe `.tema-escuro`.
- **Delegação de Eventos, Parâmetros de URL e Controle de Propagação no DOM:** Na segunda etapa, consultamos exemplos sobre o funcionamento dos métodos `event.target.closest()` e `event.target.matches()` para delegação de eventos, uso de `URLSearchParams` e `window.location.hash` para alimentar as páginas internas coringa (`noticia.html` e `vereador.html`), bem como o uso de `event.stopPropagation()` para implementar o fechamento automático do menu suspenso de Configurações ao clicar fora dele.
- **Manipulação e Codificação de Cookies HTTP:** Solicitamos explicações sobre o uso correto das funções nativas `encodeURIComponent()` e `decodeURIComponent()` junto ao parâmetro `max-age` na propriedade

`document.cookie`, evitando erros de formatação ao salvar nomes acentuados e e-mails enviados no formulário da Ouvidoria.

4. Estrutura, Execução e Considerações Finais

4.1. Estrutura do Código

O projeto está organizado no diretório principal da aplicação, dividido em arquivos com responsabilidades bem definidas: o arquivo `index.html` abriga a página principal com todas as seções institucionais, cabeçalho, barra lateral, filtros e formulário da Ouvidoria; os arquivos `noticia.html` e `vereador.html` atuam como páginas internas coringa (`data-pagina="interna"`) para renderização dinâmica das matérias completas e dos perfis parlamentares; o arquivo `style.css` concentra as variáveis visuais, layouts em Flexbox e Grid, estilos das páginas internas, regras do Modo Escuro, classes de estado dinâmico e *Media Queries*; o arquivo `script.js` reúne toda a lógica em JavaScript nativo, manipulação de eventos do DOM, leitura de parâmetros de URL e gerenciamento das 15 chaves de armazenamento no navegador; a pasta `Imagens/` armazena o brasão municipal, a imagem de fundo do cabeçalho e os retratos oficiais dos parlamentares; e o arquivo `CHANGELOG.txt` documenta o histórico de evolução das entregas parciais.

4.2. Instruções de Execução

Por utilizar exclusivamente tecnologias nativas da plataforma Web (HTML5, CSS3 e JavaScript puro), o sistema dispensa a instalação de bibliotecas externas ou gerenciadores de pacotes. Para executar o projeto, basta extrair o arquivo `.zip` mantendo todos os arquivos e a pasta `Imagens/` no mesmo diretório e abrir o arquivo `index.html` em qualquer navegador moderno (como Google Chrome, Mozilla Firefox ou Microsoft Edge). Opcionalmente, para inspecionar todos os cookies HTTP e chaves de armazenamento nas ferramentas do desenvolvedor (*F12* $\rightarrow$ aba *Aplicativo* ou *Armazenamento*), recomenda-se abrir o diretório por meio de um servidor local simples, como a extensão *Live Server* ou executando o comando `python3 -m http.server 8000` no terminal e acessando `http://localhost:8000`.

4.3. Considerações Finais

O projeto da Câmara Municipal de Buriti do Codô atingiu todos os objetivos propostos ao longo das duas etapas da disciplina. Na primeira fase foi construída uma base semântica, acessível e responsiva utilizando HTML5 e CSS3. Isso permitiu que a aplicação evoluísse de maneira limpa e estruturada na etapa final para incorporar programação orientada a eventos em JavaScript, manipulação dinâmica de elementos na árvore DOM, páginas internas coringa alimentadas por parâmetros de URL e persistência real de dados no navegador utilizando `localStorage`, `sessionStorage` e `cookies` HTTP, consolidando na prática os conceitos fundamentais do desenvolvimento *Front-end*.

Referências

[1] MOZILLA DEVELOPER NETWORK (MDN). *Web technology for developers (HTML, CSS & JavaScript)*. 2026. Disponível em: `https://developer.mozilla.org/pt-BR/`.

[2] W3C. *Web Content Accessibility Guidelines (WCAG) 2.1*. 2026. Disponível em: https://www.w3.org/TR/WCAG21/.

[3] SILVA, Evandro J. R. *Notas de Aula e Materiais da Disciplina de Programação para a Web (Aulas 01 a 15)*. Picos, PI: Universidade Federal do Piauí (UFPI), 2026.

[4] SOCIEDADE BRASILEIRA DE COMPUTAÇÃO (SBC). *Modelos para Publicação de Artigos*. Porto Alegre, RS: SBC, 2026. Disponível em: https://www.sbc.org.br/.